

Submission Worksheet

Submission Data

Course: IT202-008-S2026

Assignment: IT202 PHP Multi-Dimension Problems

Student: Stephane E. (se283)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 4/2/2026 7:57:06 PM

Updated: 4/6/2026 11:16:50 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-php-multi-dimension-problems/grading/se283>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-php-multi-dimension-problems/view/se283>

Instructions

- Overview Link: <https://youtu.be/jd5giyXReal>
- 1. Ensure you read all instructions and objectives before starting.
- 2. Create a new branch from qa called M6-Homework
 1. `git checkout qa` (ensure proper starting branch)
 2. `git pull origin qa` (ensure history is up to date)
 3. `git checkout -b M6-Homework` (create and switch to branch)
- 3. Copy the template code from here: [GitHub Repository - M6 Homework](#)
 - It includes Problems 1-3 and `base.php`. Put all into an M6 folder or similar inside your `public_html`
 - Immediately record to history
 - `git add public_html`
 - `git commit -m "adding M6 HW baseline files"`
 - `git push origin M6-Homework`
 - Create a Pull Request from M6-Homework to dev and keep it open
- 4. Fill out the below worksheet
 - Each Problem requires the following as you work
 - Ensure there's a comment with your UCID, date, and brief summary of how the problem was solved
 - Initial outline/plan of how you'll solve it via comments (add/commit after this stage)
 - Code solution (add/commit periodically as needed)
- 5. Once finished, click "Submit and Export"
- 6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 1. `git add .`
 2. `git commit -m "adding PDF"`
 3. `git push origin M6-Homework`
 4. On Github merge the pull request from M6-Homework to qa
 5. On Github create a pull request from qa to prod and immediately merge. (This will trigger the prod deploy to make the render.com prod links work)

7. Upload the same PDF to Canvas
8. Sync Local
 1. git checkout qa
 2. git pull origin qa

Section #1: (2.5 pts.) Problem 1 - Subset

Progress: 100%

≡ Task #1 (2.50 pts.) - Edit the `processBirds` function to extract properties

Progress: 100%

Details:

- Only make edits where noted via provided comments
- Challenge: Extract the name, color, region of each bird into the \$subset array
- Step 1: sketch out plan using comments (include ucid and date)
- Step 2: Add/commit your outline of comments (required for full credit)
- Step 3: Add code to solve the problem (add/commit as needed)

🖼️ Part 1:

Progress: 100%

Details:

Two screenshots are expected

1. Snippet of relevant code showing solution (with ucid/date comment)
2. Full output of executing the program (visit the proper file on Render.com qa after a manual deploy)

```

function processBirds($birds) {
    // TODO: Extract the name, color, region into a separate multi-dimensional array called $subset
    // Create $subset = [] // result array
    // Create $subset
    // Create $subset

    // Use a foreach loop to iterate through the $birds array.
    // Create a temporary associative array containing only the keys name, color, and region.
    // Append that temporary array into the $subset results array
    // $subset = array_merge($subset, $temp);

    foreach ($birds as $bird) {
        $subset[] = [
            'name' => $bird['name'],
            'color' => $bird['color'],
            'region' => $bird['region']
        ];
    }

    // Append $subset into $subset
    // $subset = array_merge($subset, $subset);

    return $subset;
}

// Test the function
$birds = [
    ['name' => 'American Robin', 'color' => 'Red', 'region' => 'North America'],
    ['name' => 'Blue Jay', 'color' => 'Blue', 'region' => 'North America'],
    ['name' => 'Scarlet Tanager', 'color' => 'Red', 'region' => 'South America'],
    ['name' => 'Pied-billed Grebe', 'color' => 'Black', 'region' => 'Africa']
];

$result = processBirds($birds);
print_r($result);
  
```

Snippet of relevant code showing solution (with ucid/date comment)

```

Array
(
    [0] => Array
        (
            [name] => American Robin
            [color] => Red
            [region] => North America
        )
    [1] => Array
        (
            [name] => Blue Jay
            [color] => Blue
            [region] => North America
        )
    [2] => Array
        (
            [name] => Scarlet Tanager
            [color] => Red
            [region] => South America
        )
    [3] => Array
        (
            [name] => Pied-billed Grebe
            [color] => Black
            [region] => Africa
        )
)
  
```

Full output of executing the program (visit the proper file on Render.com qa after a manual deploy)



Saved: 4/6/2026 11:14:38 PM

Part 2:

Progress: 100%

Details:

- Direct link to the file in the homework related branch from Github (should end in .php, , zero credit if it does not)
- Direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)

URL #1

https://github.com/se283/se283-it202-008-2026/Homework/public_html/M6/problem1.php



URL

<https://github.com/se283/se283-it202-008-2026>



URL #2

<https://se283-it202-008-prod.onrender.com/M6/problem1.php>



URL

<https://se283-it202-008-prod.onrender.com>



Saved: 4/6/2026 11:14:38 PM

Part 3:

Progress: 100%

Details:

Briefly explain how your solution works. Describe the logic, steps, or decisions your code makes to reach the result. Do not restate the problem or list requirements.

Your Response:

The code works as a filter for each collection of birds. It creates an empty array called \$subset and then uses a foreach loop to go through every bird one by one. Inside the loop, it ignores the unnecessary details of ID and size data, specifically selecting only the values for name, color, and region. These three values are placed into a new, smaller array and that array is stored into the \$subset list. When the loop finishes, the \$subset list contains only the wanted information and is then displayed.



Saved: 4/6/2026 11:14:38 PM

Section #2: (2.5 pts.) Problem 2 - Adding

Properties

Progress: 100%

☰ Task #1 (2.50 pts.) - Edit the `processCars` function to add properties

Progress: 100%

Details:

- Only make edits where noted via provided comments
- Challenge 1: Add a new property called `age` that's set from today's year and the car's year
- Challenge 2: Add a new property called `isClassic` that's true/false based on `$classic_age`
- Step 1: sketch out plan using comments (include `ucid` and `date`)
- Step 2: Add/commit your outline of comments (required for full credit)
- Step 3: Add code to solve the problem (add/commit as needed)

Part 1:

Progress: 100%

Details:

Two screenshots are expected

1. Snippet of relevant code showing solution (with ucid/date comment)
2. Full output of executing the program (visit the proper file on Render.com qa after a manual deploy)

Snippet of relevant code showing solution (with ucid/date comment)

Full output of executing the program (visit the proper file on Render.com qa after a manual deploy)



Saved: 4/6/2026 11:16:32 PM

Part 2:

Progress: 100%

Details:

- Direct link to the file in the homework related branch from Github (should end in .php, , zero credit if it does not)
- Direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)

URL #1

https://github.com/se283/se283-it202-00862026/Homework/public_html/M6/problem2.php



URL

<https://github.com/se283/se283-i>



URL #2

<https://se283-it202-008-prod.onrender.com/M6/problem2.php>



URL

<https://se283-it202-008-prod.onre>



Saved: 4/6/2026 11:16:32 PM

Part 3:

Progress: 100%

Details:

Briefly explain how your solution works. Describe the logic, steps, or decisions your code makes to reach the result. Do not restate the problem or list requirements.

Your Response:

The code first captures the current year using PHP's built-in date function and converts it into a number. It then loops through the list of cars one by one, subtracting each car's manufacturing year from the current year to calculate its age. Using this calculated age, the code performs a check to see if it is a classic using boolean values: if the age is greater than or equal to 25, it flags the car as a classic with a "true" value; otherwise, it marks it as "false." Finally, it combines these two new pieces of information with the original car data and stores them into the \$processedCars array.



Saved: 4/6/2026 11:16:32 PM

Section #3: (2.5 pts.) Problem 3 - Join

Progress: 100%

Task #1 (2.50 pts.) - Edit the `joinArrays` function to combine

two arrays based on a common key

Progress: 100%

Details:

- Only make edits where noted via provided comments
- Challenge: Combine the data in both arrays by the `userId` property
- Step 1: sketch out plan using comments (include `ucid` and `date`)
- Step 2: Add/commit your outline of comments (required for full credit)
- Step 3: Add code to solve the problem (add/commit as needed)

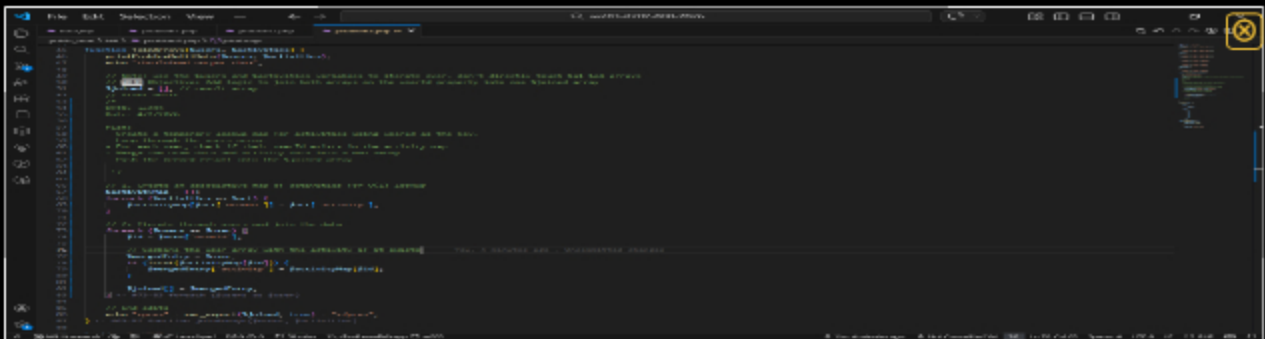
Part 1:

Progress: 100%

Details:

Two screenshots are expected

1. Snippet of relevant code showing solution (with `ucid/date` comment)
2. Full output of executing the program (visit the proper file on Render.com qa after a manual deploy)



Snippet of relevant code showing solution (with `ucid/date` comment)



Full output of executing the program (visit the proper file on Render.com qa after a manual deploy)

 Saved: 4/6/2026 11:16:50 PM

Part 2:

Progress: 100%

Details:

- Direct link to the file in the homework related branch from Github (should end in .php, , zero credit if it does not)
- Direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)

URL #1

https://github.com/se283/se283-it202-008-2026/Homework/public_html/M6/problem3.php



URL

<https://github.com/se283/se283-i>**URL #2**

<https://se283-it202-008-prod.onrender.com/M6/problem3.php>



URL

<https://se283-it202-008-prod.onre>

Saved: 4/6/2026 11:16:50 PM

⇒ Part 3:

Progress: 100%

Details:

Briefly explain how your solution works. Describe the logic, steps, or decisions your code makes to reach the result. Do not restate the problem or list requirements.

Your Response:

To solve this, the code first creates a lookup map (like a quick-reference index) from the activities list, using the userId as the key for each activity. Next, it loops through each \$user in the \$users list and uses their specific userId to obtain the matching activity from that map. Finally, it merges that activity into the user's data and saves the combined result into the final \$joined array.



Saved: 4/6/2026 11:16:50 PM

Section #4: (2.5 pts.) Misc

Progress: 100%

≡ Task #1 (0.83 pts.) - Github Details

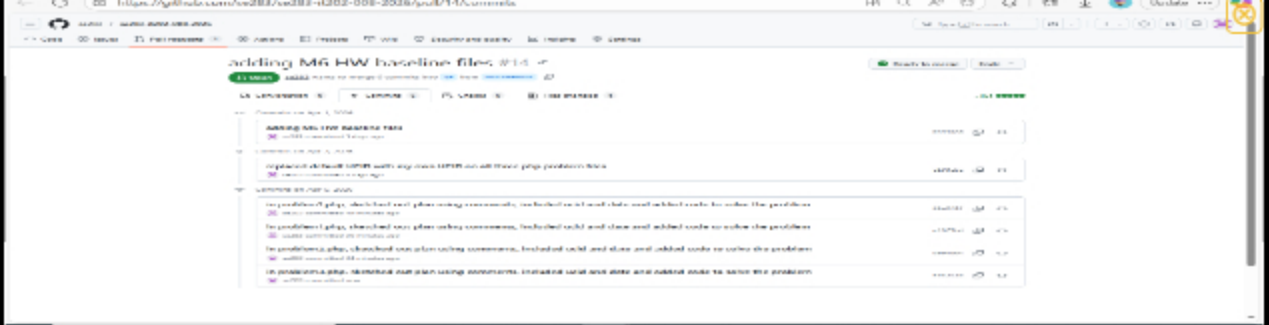
Progress: 100%

📁 Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history Following minimum should be present: addition of baseline files, comments planning thoughts, solutions



From the Commits tab of the Pull Request screenshot the commit history Following minimum should be present: addition of baseline

Saved: 4/6/2026 11:09:02 PM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in /pull/#, zero credit if it does not)

URL #1

<https://github.com/se283/se283-it202-008-2026/>



URL

<https://github.com/se283/se283-i>

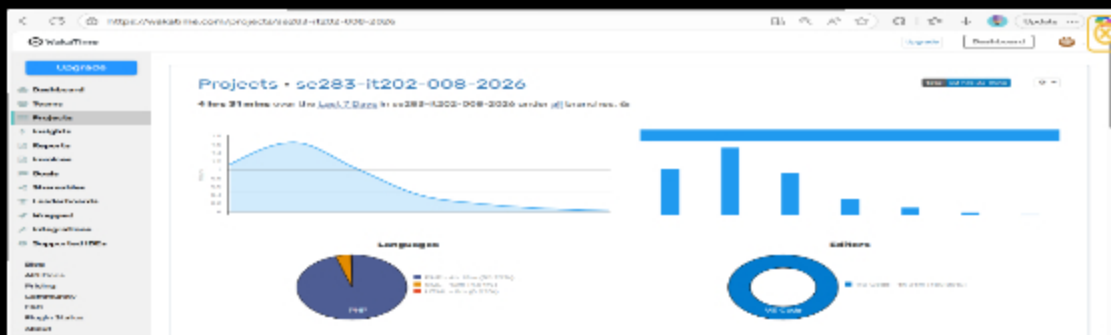
Saved: 4/6/2026 11:09:02 PM

Task #2 (0.83 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click Projects and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



Capture the overall time at the top that includes the repository name



Capture the individual time at the bottom that includes the file time



Saved: 4/6/2026 10:15:22 PM

Task #3 (0.83 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question with at least a few reasonable sentences

Your Response:

In the first PHP problem, I learned how to retrieve specific needed data while also filtering out unnecessary details from a big pile of information by reformatting the data within a loop. In the second PHP problem, I learned how to cast a string and convert it to an integer. In the third PHP problem, I learned how data from different organized lists (i.e. arrays) can be linked and brought back together using a common identifier.



Saved: 4/6/2026 10:56:01 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question with at least a few reasonable sentences

Your Response:

The easiest part of the assignment was PHP problem 1 file. This was the least difficult problem file to do. The solution to the problem, as well as the code itself, was relatively simple: used a foreach loop to go through the bird data and selectively picked out only the specific details required.



Saved: 4/6/2026 10:47:48 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question with at least a few reasonable sentences

Your Response:

The hardest part of the assignment was PHP problem 3 file. This was the most difficult problem file to do than the other problems. This is more challenging to come up a solution for, since the problem requires you to merge multiple arrays based on a single shared property: userId.



Saved: 4/6/2026 11:01:03 PM